Ex6b: Time Series Data Analysis and Forecast

Dataset: Monthly airline passengers (`airline-passengers.csv`) from https://github.com/jbrownlee/Datasets

Upload `airline-passengers.csv` to the Colab **Files** panel, or the notebook downloads it from GitHub automatically.

```
import os
import itertools
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
```

Task 1: Download the airline-passengers dataset

```
file_name = "airline-passengers.csv"
if not os.path.exists(file_name):  # file not uploaded -> download from GitHub (raw version of the given link)
    file_name = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
print("Reading data from:", file_name)
```

```
Reading data from: airline-passengers.csv
```

Task 2: Read the CSV with `parse_dates` and `index_col` so the index becomes a DatetimeIndex

```
df = pd.read_csv(file_name, parse_dates=['Month'], index_col='Month')
df = df.asfreq('MS')          # monthly data, each value at the start of the month
series = df['Passengers']

print("Index type:", type(df.index).__name__)
print("Index frequency:", df.index.freqstr)
display(df.head())
```

```
Index type: DatetimeIndex
Index frequency: MS
```

	Passengers
Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Task 3: Explore the Dataset

```
print("First five records:")
display(df.head())
print("Last five records:")
display(df.tail())

print("Shape:", df.shape)
print("Date range:", df.index.min().date(), "to", df.index.max().date())
print("\nDataset information:")
df.info()
```

```
First five records:
```

	Passengers
Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129

Month	Passengers
1949-05-01	121

Last five records:

Month	Passengers
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

```
Shape: (144, 1)
Date range: 1949-01-01 to 1960-12-01

Dataset information:
<class 'pandas.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Freq: MS
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
```

```
print("Summary statistics:")
display(df.describe().T)
print("Missing values:", df.isnull().sum().sum())
print("Duplicate dates:", df.index.duplicated().sum())

print("\nAverage passengers per year:")
display(series.groupby(series.index.year).mean().round(1).to_frame('Mean Passengers').T)
```

Summary statistics:

	count	mean	std	min	25%	50%	75%	max
Passengers	144.0	280.298611	119.966317	104.0	180.0	265.5	360.5	622.0

```
Missing values: 0
Duplicate dates: 0
```

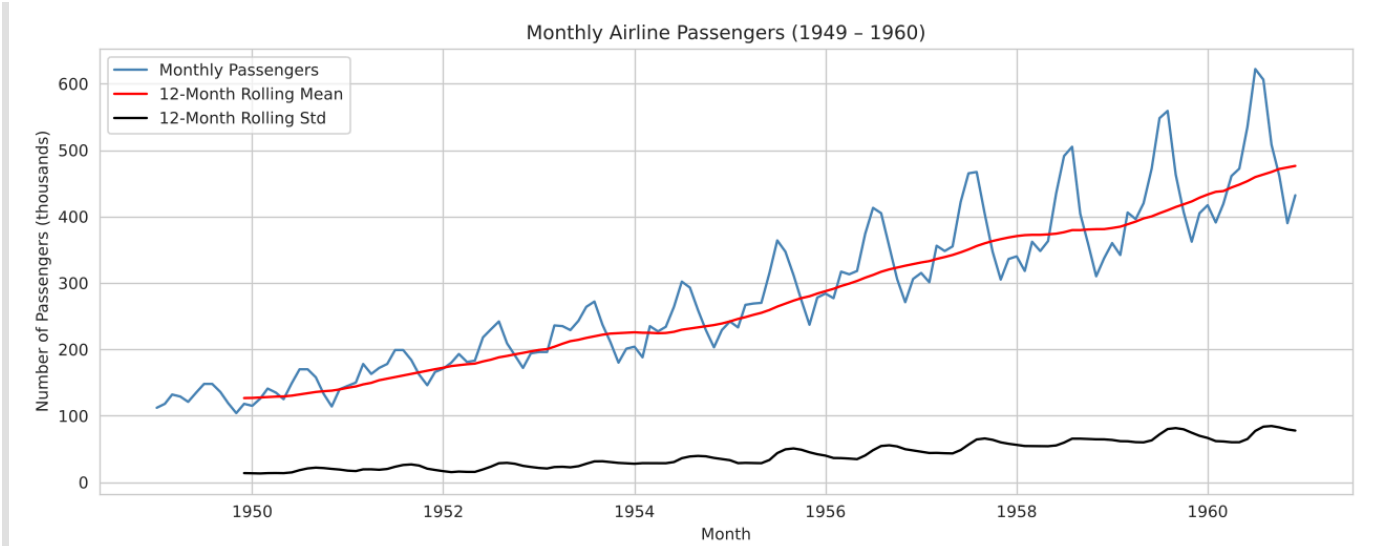
Average passengers per year:

Month	1949	1950	1951	1952	1953	1954	1955	1956	1957
Mean Passengers	126.7	139.7	170.2	197.0	225.0	238.9	284.0	328.2	368.4

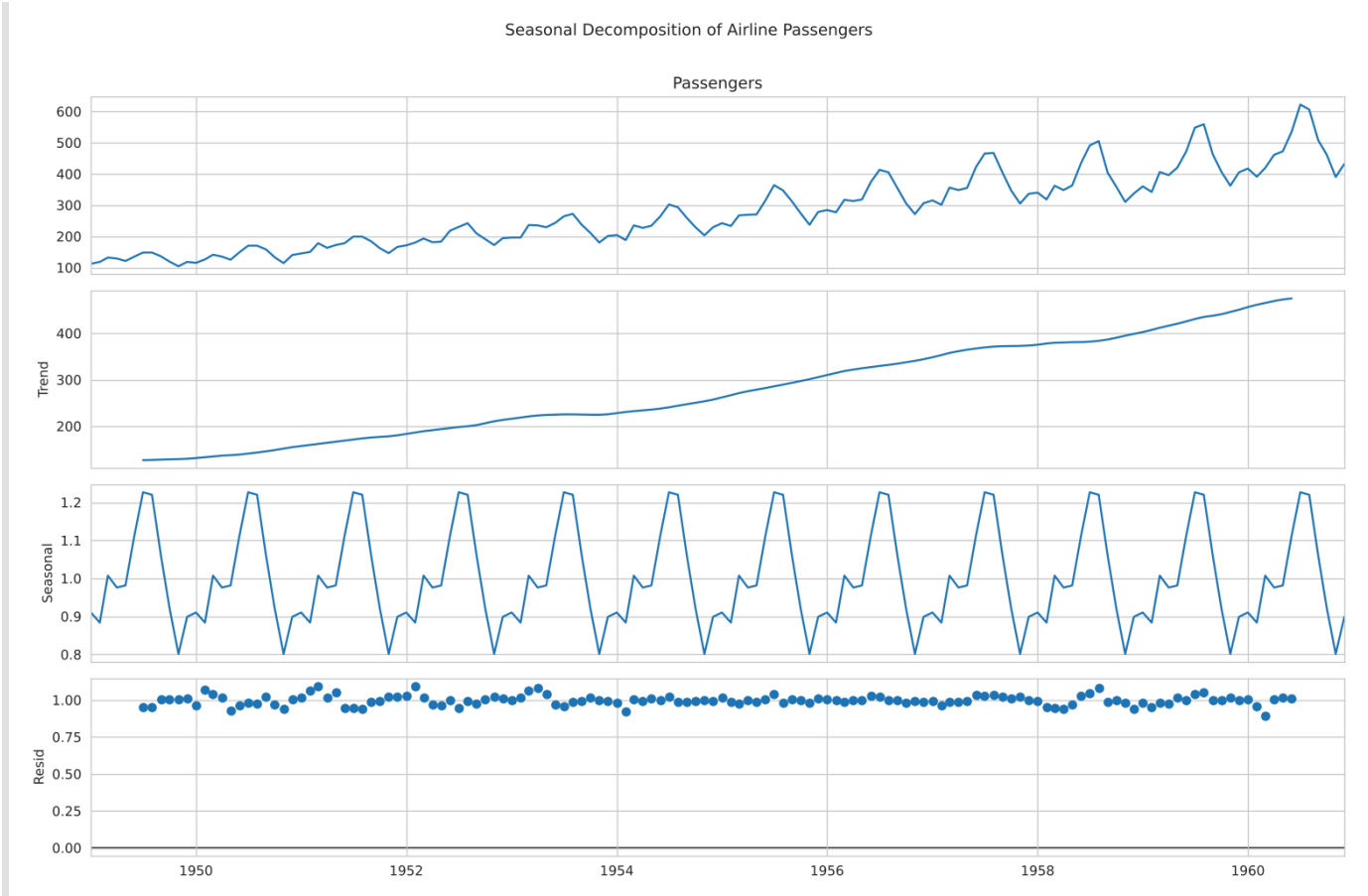
Month	1958	1959	1960
Mean Passengers	381.0	428.3	476.2

Task 4: Plot the series to inspect trend / seasonality / outliers

```
plt.figure(figsize=(14, 5))
plt.plot(series, label='Monthly Passengers', color='steelblue')
plt.plot(series.rolling(window=12).mean(), label='12-Month Rolling Mean', color='red')
plt.plot(series.rolling(window=12).std(), label='12-Month Rolling Std', color='black')
plt.title('Monthly Airline Passengers (1949 – 1960)')
plt.xlabel('Month')
plt.ylabel('Number of Passengers (thousands)')
plt.legend()
plt.show()
```




```
decomposition = seasonal_decompose(series, model='multiplicative', period=12)
fig = decomposition.plot()
fig.set_size_inches(14, 9)
fig.suptitle('Seasonal Decomposition of Airline Passengers', y=1.01)
plt.tight_layout()
plt.show()
```



Observation: The series has a clear **upward trend** and a strong **yearly seasonality** (peaks every July–August). The size of the seasonal swings grows with the level of the series. No unusual outliers are visible.

Task 5: Stationarity check – Augmented Dickey-Fuller (ADF) test

```
def adf_test(data, title):
    result = adfuller(data.dropna(), autolag='AIC')
    print(f"ADF Test — {title}")
    print(f"  ADF Statistic       : {result[0]:.4f}")
    print(f"  p-value             : {result[1]:.6f}")
    print(f"  Lags used           : {result[2]}")
    print(f"  Number of observations: {result[3]}")
    for key, value in result[4].items():
        print(f"  Critical Value ({key}): {value:.4f}")
```

```
    if result[1] <= 0.05:
        print("  Result: p-value <= 0.05 → reject H0 → the series is STATIONARY\n")
    else:
        print("  Result: p-value > 0.05 → fail to reject H0 → the series is NON-STATIONARY\n")
    return result[1]

p_original = adf_test(series, 'Original Series')
```

```
ADF Test — Original Series
  ADF Statistic       : 0.8154
  p-value             : 0.991880
  Lags used           : 13
  Number of observations: 130
  Critical Value (1%): -3.4817
  Critical Value (5%): -2.8840
  Critical Value (10%): -2.5788
  Result: p-value > 0.05 → fail to reject H0 → the series is NON-STATIONARY
```

Task 6: Convert to stationary by differencing (first difference, d = 1)

```
diff1 = series.diff().dropna()
display(diff1.head().to_frame('First Difference'))
```

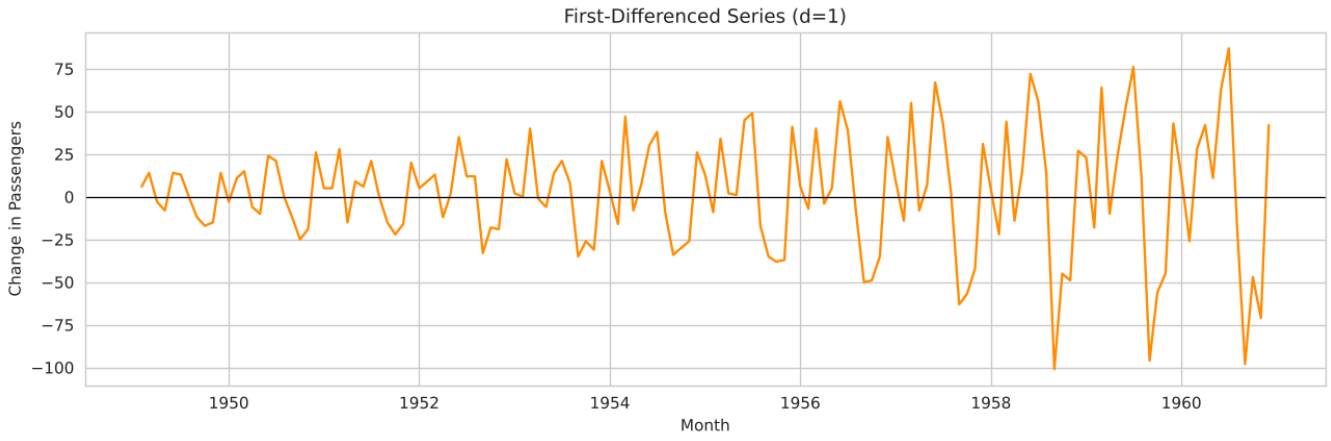
Month	First Difference
1949-02-01	6.0
1949-03-01	14.0
1949-04-01	-3.0
1949-05-01	-8.0
1949-06-01	14.0

Task 7: Re-run ADF on the differenced series and visualise it

```
p_diff1 = adf_test(diff1, 'First Difference (d=1)')

plt.figure(figsize=(14, 4))
plt.plot(diff1, color='darkorange')
plt.axhline(0, color='black', linewidth=0.8)
plt.title('First-Differenced Series (d=1)')
plt.xlabel('Month')
plt.ylabel('Change in Passengers')
plt.show()
```

```
ADF Test — First Difference (d=1)
  ADF Statistic       : -2.8293
  p-value             : 0.054213
  Lags used           : 12
  Number of observations: 130
  Critical Value (1%): -3.4817
  Critical Value (5%): -2.8840
  Critical Value (10%): -2.5788
  Result: p-value > 0.05 → fail to reject H0 → the series is NON-STATIONARY
```



Task 8: Still non-stationary? Try a second difference (d = 2) or a log transformation before differencing

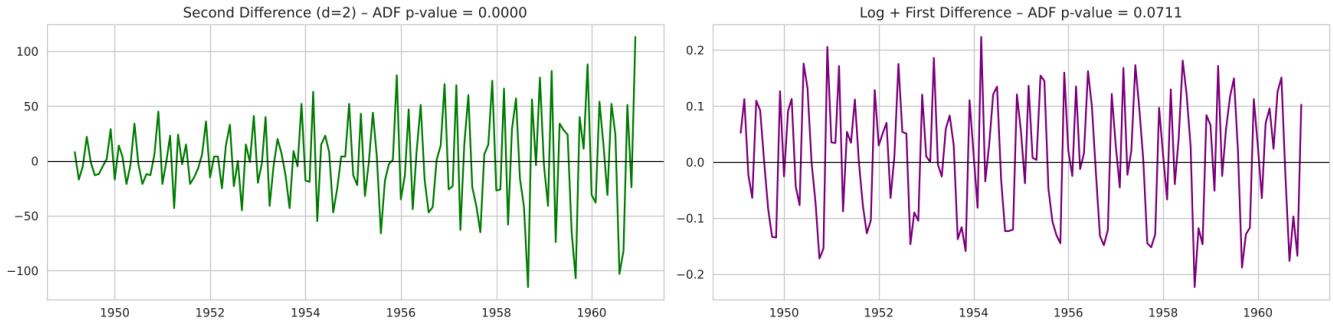
```
diff2 = series.diff().diff().dropna()
log_diff1 = np.log(series).diff().dropna()
```

```
p_diff2 = adf_test(diff2, 'Second Difference (d=2)')
p_log_diff1 = adf_test(log_diff1, 'Log Transformation + First Difference')

fig, axes = plt.subplots(1, 2, figsize=(16, 4))
axes[0].plot(diff2, color='green')
axes[0].axhline(0, color='black', linewidth=0.8)
axes[0].set_title(f'Second Difference (d=2) — ADF p-value = {p_diff2:.4f}')
axes[1].plot(log_diff1, color='purple')
axes[1].axhline(0, color='black', linewidth=0.8)
axes[1].set_title(f'Log + First Difference — ADF p-value = {p_log_diff1:.4f}')
plt.tight_layout()
plt.show()
```

```
ADF Test — Second Difference (d=2)
  ADF Statistic       : -16.3842
  p-value             : 0.000000
  Lags used           : 11
  Number of observations: 130
  Critical Value (1%): -3.4817
  Critical Value (5%): -2.8840
  Critical Value (10%): -2.5788
  Result: p-value <= 0.05 → reject H0 → the series is STATIONARY

ADF Test — Log Transformation + First Difference
  ADF Statistic       : -2.7171
  p-value             : 0.071121
  Lags used           : 14
  Number of observations: 128
  Critical Value (1%): -3.4825
  Critical Value (5%): -2.8844
  Critical Value (10%): -2.5790
  Result: p-value > 0.05 → fail to reject H0 → the series is NON-STATIONARY
```

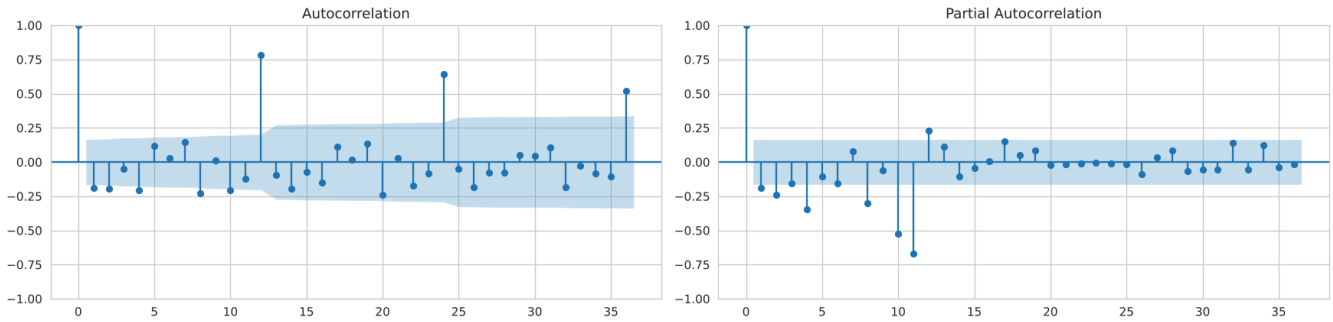



```
if p_original <= 0.05:
    d = 0
elif p_diff1 <= 0.05:
    d = 1
else:
    d = 2
print(f"Original: p = {p_original:.4f} | d=1: p = {p_diff1:.4f} | d=2: p = {p_diff2:.6f} | log+d=1: p = {p_log_diff1:.4f}")
print(f"Number of differences needed to make the series stationary: d = {d}")
```

```
Original: p = 0.9919 | d=1: p = 0.0542 | d=2: p = 0.000000 | log+d=1: p = 0.0711
Number of differences needed to make the series stationary: d = 2
```

ACF and PACF of the stationary (differenced) series – the spikes at lag 12 confirm the yearly seasonality, so the ARIMA model needs autoregressive terms up to lag 12 to capture it.

```
stationary = series.diff(1) if d == 1 else series.diff().diff()
fig, axes = plt.subplots(1, 2, figsize=(16, 4))
plot_acf(stationary.dropna(), lags=36, ax=axes[0])
plot_pacf(stationary.dropna(), lags=36, ax=axes[1], method='ywm')
plt.tight_layout()
plt.show()
```



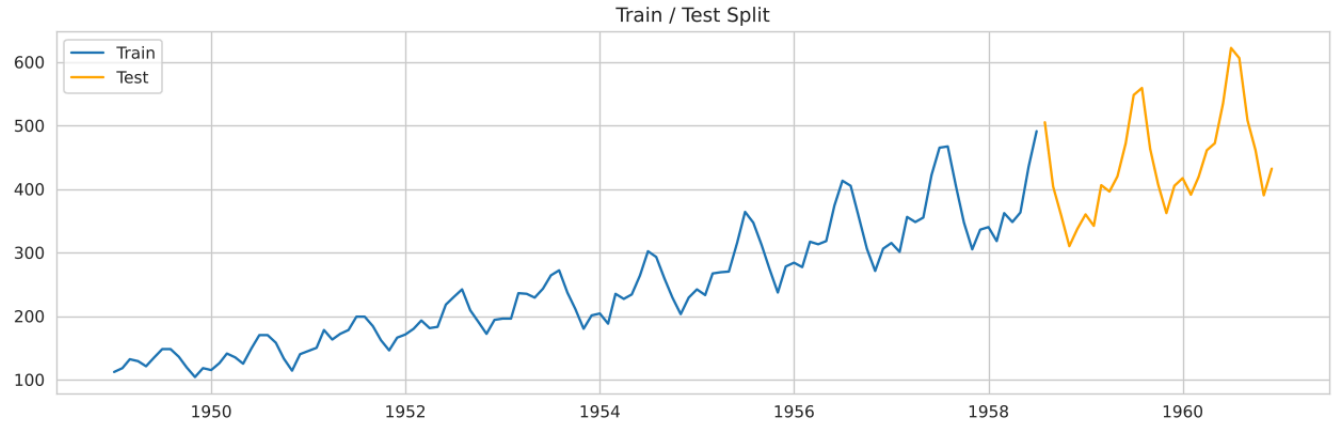
Task 9: Split the original series into training and testing sets (80% / 20%, in time order)

```
train_size = int(len(series) * 0.8)
train = series.iloc[:train_size]
test = series.iloc[train_size:]

print(f"Training set: {len(train)} months ({train.index.min().date()} to {train.index.max().date()})")
print(f"Testing set : {len(test)} months ({test.index.min().date()} to {test.index.max().date()})")

plt.figure(figsize=(14, 4))
plt.plot(train, label='Train')
plt.plot(test, label='Test', color='orange')
plt.title('Train / Test Split')
plt.legend()
plt.show()
```

```
Training set: 115 months (1949-01-01 to 1958-07-01)
Testing set : 29 months (1958-08-01 to 1960-12-01)
```



Task 10: Fit the ARIMA model on the training set (d from Task 8)

The AR order `p` (0–12) and MA order `q` (0–2) are selected by the lowest AIC on the training data.

```
results = []
for p, q in itertools.product(range(0, 13), range(0, 3)):
    try:
        fit = ARIMA(train, order=(p, d, q)).fit()
        results.append((p, d, q, fit.aic))
    except Exception:
        continue

aic_df = pd.DataFrame(results, columns=['p', 'd', 'q', 'AIC']).sort_values('AIC').reset_index(drop=True)
print("Top 5 ARIMA orders by AIC:")
display(aic_df.head())

best_order = tuple(int(v) for v in aic_df.loc[0, ['p', 'd', 'q']])
print("Selected ARIMA order (p, d, q):", best_order)
```

```
Top 5 ARIMA orders by AIC:
```

	p	d	q	AIC
0	12	2	1	871.251625
1	12	2	0	871.561337
2	11	2	1	871.979560
3	11	2	2	873.615894
4	12	2	2	874.516879

```
Selected ARIMA order (p, d, q): (12, 2, 1)
```

```
model = ARIMA(train, order=best_order)
model_fit = model.fit()
print(model_fit.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:              Passengers   No. Observations:                  115
Model:                ARIMA(12, 2, 1)   Log Likelihood                -421.626
Date:                Fri, 09 Oct 2026   AIC                            871.252
Time:                        05:39:09   BIC                            909.435
Sample:                    01-01-1949   HQIC                           886.746
                         - 07-01-1958
Covariance Type:                  opg
```

```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.1446      0.105     -1.374      0.169      -0.351       0.062
ar.L2         -0.2432      0.108     -2.248      0.025      -0.455      -0.031
ar.L3         -0.1503      0.106     -1.414      0.157      -0.359       0.058
ar.L4         -0.2543      0.106     -2.390      0.017      -0.463      -0.046
ar.L5         -0.1356      0.098     -1.384      0.166      -0.328       0.056
ar.L6         -0.2355      0.103     -2.288      0.022      -0.437      -0.034
ar.L7         -0.1746      0.107     -1.631      0.103      -0.384       0.035
ar.L8         -0.2773      0.118     -2.360      0.018      -0.508      -0.047
ar.L9         -0.1644      0.110     -1.492      0.136      -0.380       0.052
ar.L10        -0.2838      0.101     -2.811      0.005      -0.482      -0.086
ar.L11        -0.1217      0.121     -1.008      0.314      -0.358       0.115
ar.L12         0.6620      0.111      5.988      0.000       0.445       0.879
ma.L1         -0.9668      0.057    -16.978      0.000      -1.078      -0.855
sigma2        83.2190     13.164      6.322      0.000      57.418     109.020
==============================================================================
Ljung-Box (L1) (Q):                   1.61   Jarque-Bera (JB):                 2.87
Prob(Q):                              0.20   Prob(JB):                         0.24
Heteroskedasticity (H):               1.76   Skew:                             0.39
Prob(H) (two-sided):                  0.08   Kurtosis:                         2.99
==============================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Task 11: Forecast on the test period

```
predictions = model_fit.forecast(steps=len(test))
predictions.index = test.index

forecast_df = pd.DataFrame({'Actual': test, 'Predicted': predictions.round(2),
                            'Error': (test - predictions).round(2)})
display(forecast_df)
```

Month	Actual	Predicted	Error
1958-08-01	505	495.36	9.64
1958-09-01	404	439.47	-35.47
1958-10-01	359	384.04	-25.04
1958-11-01	310	345.28	-35.28
1958-12-01	337	374.09	-37.09
1959-01-01	360	373.20	-13.20
1959-02-01	342	351.33	-9.33
1959-03-01	406	386.04	19.96
1959-04-01	396	371.95	24.05
1959-05-01	420	390.69	29.31
1959-06-01	472	462.70	9.30
1959-07-01	548	518.11	29.89
1959-08-01	559	524.42	34.58
1959-09-01	463	474.93	-11.93
1959-10-01	407	421.36	-14.36
1959-11-01	362	386.12	-24.12
1959-12-01	405	412.60	-7.60
1960-01-01	417	407.52	9.48
1960-02-01	391	385.21	5.79
1960-03-01	419	411.32	7.68
1960-04-01	461	396.82	64.18
1960-05-01	472	418.54	53.46
1960-06-01	535	490.08	44.92
1960-07-01	622	544.81	77.19
1960-08-01	606	553.05	52.95
1960-09-01	508	509.47	-1.47
1960-10-01	461	458.27	2.73
1960-11-01	390	426.82	-36.82
1960-12-01	432	451.10	-19.10

Task 12: Compute MSE, MAE and RMSE

```
mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
```

```
rmse = np.sqrt(mse)

print(f"ARIMA{best_order} forecast accuracy on the test set:")
print(f"  MSE  = {mse:.4f}")
print(f"  MAE  = {mae:.4f}")
print(f"  RMSE = {rmse:.4f}")
```

```
ARIMA(12, 2, 1) forecast accuracy on the test set:
  MSE  = 1018.2028
  MAE  = 25.7213
  RMSE = 31.9093
```

Task 13: Plot train, test and predictions in a single plot

```
plt.figure(figsize=(14, 6))
plt.plot(train, label='Train', color='steelblue')
plt.plot(test, label='Test (Actual)', color='orange')
plt.plot(predictions, label=f'Predictions ARIMA{best_order}', color='red', linestyle='--')
plt.axvline(test.index[0], color='grey', linestyle=':')
plt.title(f'Airline Passengers – ARIMA{best_order} Forecast  (RMSE = {rmse:.2f})')
plt.xlabel('Month')
plt.ylabel('Number of Passengers (thousands)')
plt.legend()
plt.show()
```

